```
import { existsSync, readFileSync, writeFileSync } from "node:fs";

interface Listing {
  id: string;
  source: "vinted" | "ebay";
  title: string;
  price: number;
  currency: string;
  url: string;
  image?: string;
  extra?: string;
}

interface Search { name: string; query: string; maxPrice?: number; lockedMax?: nu

const config = JSON.parse(readFileSync("samsung.json", "utf8")) as {
  intervalSeconds: number; exclude: string[]; searches: Search[];
  fast?: { enabled?: boolean; query?: string; queries?: string[]; everySeconds?:
};
// ⚡ Mode rapide : UNE recherche large « samsung » (plus récentes d'abord) relue
// Chaque nouvelle annonce passe ensuite par les mêmes filtres et les mêmes plafo
const FAST = {
  enabled: config.fast?.enabled ?? true,
  query: config.fast?.query ?? "samsung",
  queries: config.fast?.queries?.length ? config.fast.queries : [config.fast?.que
  everySeconds: Math.max(config.fast?.everySeconds ?? 8, 5),
  priceTo: config.fast?.priceTo ?? Math.max(...config.searches.map((x) => x.maxPr
  domains: (config.fast?.domains?.length ? config.fast.domains : ["www.vinted.fr"
};

const sleep = (ms: number) => new Promise((r) => setTimeout(r, ms));
const norm = (s: string) => s.normalize("NFD").replace(/[\u0300-\u036f]/g, "").to

// ---------- Plafonds par modèle (lus dans config.json) ----------
// Modèles qui se revendent plus de 150 € : Google Pixel 7 Pro à 10, Galaxy S, No
const modelKey = (t: string): string | null => {
  const s = t.replace(/\+/g, " plus ");
  const px = s.match(/\bpixel\s*(10|[7-9])\s*(pro\s*xl|pro\s*fold|pro|a)?(?![0-9]
  if (px) return `pixel${px[1]}${px[2] ? " " + px[2].replace(/\s+/g, " ") : ""}`;
  if (s.includes("pixel")) return null;
  let m = s.match(/\bz?\s*fold\s*([3-7])\b/);
  if (m) return `fold${m[1]}`;
  m = s.match(/\bz?\s*flip\s*([3-7])\b/);
```

```
  if (m) return `flip${m[1]}`;
  m = s.match(/\ba\s?(3[67]|5[5-7])\b/);
  if (m) return `a${m[1]}`;
  m = s.match(/\bnote\s*(20)\s*(ultra)?/);
  if (m) return `note20${m[2] ? " ultra" : ""}`;
  m = s.match(/\bs\s?(2[0-6])\s*(ultra|u\b|plus|fe|edge)?/);
  if (m) {
    const v = m[2] === "u" ? "ultra" : (m[2] ?? "");
    return `s${m[1]} ${v}`.trim();
  }
  return null;
};
const modelCap = new Map<string, number>();
const modelResale = new Map<string, number>();
const modelLocked = new Map<string, number>();
for (const s of config.searches) {
  { const k = modelKey(norm(s.query)); if (k && s.resale) modelResale.set(k, s.re
  if (!s.maxPrice) continue;
  const k = modelKey(norm(s.query));
  if (k) modelCap.set(k, s.maxPrice);
}
console.log(`Plafonds par modèle chargés : ${modelCap.size}`);

const SEEN_FILE = "seen-samsung.json";
const seen = new Set<string>(existsSync(SEEN_FILE) ? JSON.parse(readFileSync(SEEN
const saveSeen = () => writeFileSync(SEEN_FILE, JSON.stringify([...seen].slice(-8
const rejected = new Set<string>();

const COLORS = { vinted: 0x09b1ba, ebay: 0xe53238 } as const;

async function sendToDiscord(l: Listing, searchName: string, retry = 0): Promise<
  const url = process.env.DISCORD_WEBHOOK_SAMSUNG;
  if (!url) throw new Error("DISCORD_WEBHOOK_SAMSUNG manquant");
  const body = {
    username: "Samsung Scanner",
    embeds: [{
      title: l.title.slice(0, 250),
      url: l.url,
      color: 0x1428a0,
      fields: [
        { name: "💰 Prix", value: `**${l.price} ${l.currency}**`, inline: true },
        { name: "🛒 Source", value: l.source.toUpperCase(), inline: true },
        { name: "🔎 Recherche", value: searchName, inline: true },
        ...(l.extra ? [{ name: "ℹ️ Infos", value: l.extra, inline: false }] : [])
        ...((() => {
          const t = norm(l.title);
```

```
          if (LOCKED.test(t) && !UNLOCKED.test(t)) return [{ name: "🔒 Bloqué com
          const r = modelResale.get(modelKey(t) ?? "");
          return r && l.price ? [{ name: "📈 Revente estimée", value: `~${r} € ·
        })()),
      ],
      ...(l.image ? { image: { url: l.image } } : {}),
      timestamp: new Date().toISOString(),
    }],
  };
  const r = await fetch(url, {
    method: "POST",
    headers: { "content-type": "application/json" },
    body: JSON.stringify(body),
  });
  if (r.status === 429 && retry < 3) {
    const j: any = await r.json().catch(() => ({}));
    await sleep(((j.retry_after ?? 2) as number) * 1000 + 200);
    return sendToDiscord(l, searchName, retry + 1);
  }
  if (!r.ok) throw new Error(`Discord HTTP ${r.status}`);
}

// ---------- Vinted (lecture de la page de recherche, plusieurs pays possibles)
const UA = "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,
const VBASE = "https://www.vinted.fr";
const PHONES_CATALOG = "3661"; // catégorie Vinted « Téléphones portables »

// Une session (cookies + pause) par pays
interface VSession { base: string; jar: Map<string, string>; blockedUntil: number
const sessions = new Map<string, VSession>();
const sessionFor = (host: string): VSession => {
  let v = sessions.get(host);
  if (!v) { v = { base: `https://${host}`, jar: new Map(), blockedUntil: 0, strik
  return v;
};
const countryOf = (base: string) => base.split(".").pop()!.toUpperCase();

function storeCookies(v: VSession, res: Response) {
  for (const c of res.headers.getSetCookie()) {
    const [pair] = c.split(";");
    const i = pair.indexOf("=");
    if (i > 0) v.jar.set(pair.slice(0, i).trim(), pair.slice(i + 1));
  }
}

const vHeaders = (v: VSession, extra: Record<string, string> = {}): Record<string
```

```
  "user-agent": UA,
  accept: "text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8",
  "accept-language": "fr-FR,fr;q=0.9,en;q=0.7",
  "cache-control": "no-cache",
  ...(v.jar.size ? { cookie: [...v.jar].map(([k, val]) => `${k}=${val}`).join(";
  ...extra,
});

async function refreshSession(v: VSession) {
  const h = await fetch(v.base + "/", { headers: vHeaders(v), redirect: "manual"
  storeCookies(v, h);
  await h.text();
  const r = await fetch(v.base + "/web/api/auth/refresh", {
    method: "POST",
    headers: vHeaders(v, { accept: "application/json", "content-type": "applicati
    body: "{}",
    redirect: "manual",
  });
  storeCookies(v, r);
  await r.text();
  console.log(`Vinted ${countryOf(v.base)} session rafraîchie : HTTP ${r.status}`
}

async function getCatalogHtml(v: VSession, u: URL): Promise<string> {
  for (let attempt = 0; attempt < 3; attempt++) {
    const r = await fetch(u, { headers: vHeaders(v), redirect: "manual" });
    storeCookies(v, r);
    if (r.status === 403 || r.status === 429) {
      // pause progressive : 1 min, 2 min, 5 min, puis 10 min
      v.strikes++;
      const mins = [1, 2, 5, 10][Math.min(v.strikes - 1, 3)];
      v.blockedUntil = Date.now() + mins * 60_000;
      v.jar.clear();
      throw new Error(`Vinted ${countryOf(v.base)} bloque (HTTP ${r.status}) : pa
    }
    if (r.status >= 300 && r.status < 400) {
      await refreshSession(v);
      continue;
    }
    if (!r.ok) throw new Error(`Vinted HTTP ${r.status}`);
    const html = await r.text();
    if (/<title>\s*Session refresh/i.test(html)) {
      await refreshSession(v);
      continue;
    }
    v.strikes = 0;
```

```
    return html;
  }
  throw new Error("Vinted : session non valide");
}

const decode = (s: string) =>
  s.replace(/&amp;/g, "&").replace(/&quot;/g, '"').replace(/&#39;|&apos;/g, "'").

const SP = "(?:\\s|\\u00a0|\\u202f| | | )*";
const PRICE_RE = new RegExp("(\\d{1,5}(?:[.,]\\d{1,2})?)" + SP + "(?:€|&euro;|&#8
const STATE_RE = /(Neuf avec étiquette|Neuf sans étiquette|Très bon état|Bon état
const IMG_RE = /https:\/\/images\d*\.vinted\.net\/[^"'\s)<>\\]+/;

function parseCatalog(html: string, base = VBASE): Listing[] {
  const out: Listing[] = [];
  const ids = new Set<string>();

  // 1) Cartes de la page (liens <a href="/items/123-titre">)
  const anchors: { id: string; slug: string; idx: number; tag: string }[] = [];
  const reA = /<a\b[^>]*?href=["'](?:https:\/\/www\.vinted\.[a-z.]+)?\/items\/(\d
  for (const m of html.matchAll(reA)) {
    if (ids.has(m[1])) continue;
    ids.add(m[1]);
    anchors.push({ id: m[1], slug: m[2], idx: m.index ?? 0, tag: m[0] });
  }
  if (anchors.length >= 3) {
    anchors.forEach((a, i) => {
      const end = Math.min(anchors[i + 1]?.idx ?? a.idx + 6000, a.idx + 6000);
      const seg = html.slice(a.idx, end);
      const p = seg.match(PRICE_RE);
      if (!p) return;
      const attr = a.tag.match(/title=["']([^"']{3,300})["']/);
      const img = seg.match(IMG_RE);
      const st = seg.match(STATE_RE);
      out.push({
        id: `vinted:${a.id}`,
        source: "vinted",
        title: attr ? decode(attr[1]) : a.slug.replace(/-/g, " "),
        price: Number(p[1].replace(",", ".")),
        currency: "EUR",
        url: `${base}/items/${a.id}-${a.slug}`,
        image: img?.[0],
        extra: st?.[1],
      });
    });
    if (out.length) return out;
```

```
  }

  // 2) Secours : données JSON de la page
  ids.clear();
  for (const m of html.matchAll(/\/items\/(\d{6,})-([a-z0-9-]*)/g)) {
    if (ids.has(m[1])) continue;
    ids.add(m[1]);
    const idx = m.index ?? 0;
    const win = html.slice(Math.max(0, idx - 1500), idx + 300).replace(/\\"/g, '"
    const all = [...win.matchAll(/"amount"\s*:\s*"?(\d+(?:\.\d+)?)/g)];
    if (!all.length) continue;
    const t = win.match(/"title"\s*:\s*"([^"]{3,200})"/g);
    const lastTitle = t ? t[t.length - 1].replace(/^"title"\s*:\s*"/, "").replace
    out.push({
      id: `vinted:${m[1]}`,
      source: "vinted",
      title: lastTitle ?? m[2].replace(/-/g, " "),
      price: Number(all[all.length - 1][1]),
      currency: "EUR",
      url: `${base}/items/${m[1]}-${m[2]}`,
      image: win.match(IMG_RE)?.[0],
    });
  }
  return out;
}

async function searchVinted(query: string, maxPrice?: number, host = "www.vinted.
  const v = sessionFor(host);
  if (Date.now() < v.blockedUntil) throw new Error(`en pause (Vinted ${countryOf(
  const u = new URL(v.base + "/catalog");
  u.searchParams.set("search_text", query);
  u.searchParams.set("catalog[]", PHONES_CATALOG); // uniquement la catégorie tél
  u.searchParams.set("order", "newest_first");
  u.searchParams.set("per_page", "96");
  if (page > 1) u.searchParams.set("page", String(page));
  u.searchParams.set("time", String(Math.floor(Date.now() / 1000))); // évite les
  if (maxPrice) u.searchParams.set("price_to", String(maxPrice));
  const html = await getCatalogHtml(v, u);
  const items = parseCatalog(html, v.base);
  if (!quiet) console.log(`🔎 ${countryOf(v.base)} ${query}${page > 1 ? ` p${page
  if (items.length === 0) {
    const k = html.indexOf("/items/");
    console.error("Aucune annonce lisible. Extrait :", html.slice(Math.max(0, k -
  }
  return items;
}
```

```
// ---------- eBay (optionnel) ----------
const EBAY_ENABLED = Boolean(process.env.EBAY_CLIENT_ID && process.env.EBAY_CLIEN
let token = "";
let tokenExp = 0;

async function ebayToken(): Promise<string> {
  if (token && Date.now() < tokenExp) return token;
  const auth = Buffer.from(`${process.env.EBAY_CLIENT_ID}:${process.env.EBAY_CLIE
  const r = await fetch("https://api.ebay.com/identity/v1/oauth2/token", {
    method: "POST",
    headers: { "content-type": "application/x-www-form-urlencoded", authorization
    body: "grant_type=client_credentials&scope=https%3A%2F%2Fapi.ebay.com%2Foauth
  });
  if (!r.ok) throw new Error(`eBay token HTTP ${r.status}`);
  const j: any = await r.json();
  token = j.access_token;
  tokenExp = Date.now() + (j.expires_in - 120) * 1000;
  return token;
}

async function searchEbay(query: string, maxPrice?: number): Promise<Listing[]> {
  if (!EBAY_ENABLED) return [];
  const u = new URL("https://api.ebay.com/buy/browse/v1/item_summary/search");
  u.searchParams.set("q", query);
  u.searchParams.set("sort", "newlyListed");
  u.searchParams.set("limit", "30");
  const filters = ["priceCurrency:EUR"];
  if (maxPrice) filters.unshift(`price:[..${maxPrice}]`);
  u.searchParams.set("filter", filters.join(","));
  const r = await fetch(u, {
    headers: { authorization: `Bearer ${await ebayToken()}`, "X-EBAY-C-MARKETPLAC
  });
  if (!r.ok) throw new Error(`eBay HTTP ${r.status}`);
  const data: any = await r.json();
  return (data.itemSummaries ?? []).map((it: any): Listing => ({
    id: `ebay:${it.itemId}`,
    source: "ebay",
    title: it.title,
    price: Number(it.price?.value),
    currency: it.price?.currency ?? "EUR",
    url: it.itemWebUrl,
    image: it.image?.imageUrl,
    extra: [it.condition, it.itemLocation?.country].filter(Boolean).join(" · ") |
  }));
}
```

```
// ---------- Filtres ----------
const MIN_PRIX = 40;

// Samsung / Pixel : on veut des téléphones qui MARCHENT (petites rayures OK), ja
// Bloqué compte Google / Samsung : accepté uniquement pour pièces, à prix très b
const LOCKED = /(compte google|bloque google|bloque compte|frp|verrouill|compte s
const UNLOCKED = /(sans compte|compte (google|samsung) (supprime|retire|deconnect
// Jamais : téléphones signalés volés, trouvés ou perdus (ils n'appartiennent pas
const STOLEN = /(blacklist|black list|vole|volé|trouve|trouvé|perdu|imei bloque|i
const BROKEN = /(casse|cassé|fissur|felure|fêlure|brise|brisé|\bhs\b|hors service

// Pièces et accessoires : on regarde seulement le DÉBUT du titre (toutes langues
const PART_START = /^(ecran|batterie|boite|boitier|vitre|dalle|lcd|oled|camera|ch

// Retourne la raison du refus, ou null si l'annonce est acceptée
const refusal = (l: Listing, s: Search): string | null => {
  const t = norm(l.title);
  const title = t.split(/,\s*marque\s*:/)[0].trim(); // titre réel de l'annonce
  if (!t.includes("samsung") && !t.includes("galaxy") && !t.includes("pixel")) re
  if (/iphone|apple|xiaomi|redmi|huawei|oppo|oneplus|honor|realme/.test(title)) r
  if (/\b(tab|watch|buds|fit)\b/.test(title)) return "pas un téléphone";
  if (PART_START.test(title)) return "pièce ou accessoire";
  const locked = LOCKED.test(t) && !UNLOCKED.test(t);
  if (!locked && norm(l.extra ?? "").includes("satisfaisant")) return "état satis
  if (STOLEN.test(t)) return "signalé volé, trouvé ou perdu";
  if (!locked && BROKEN.test(t)) return "cassé ou en panne";
  if (l.price < (locked ? 15 : MIN_PRIX)) return `prix trop bas (${l.price}€)`;

  // Plafond : le plus bas entre la recherche et le modèle réellement détecté
  const mf = t.match(/modele\s*:\s*([^,]+)/);
  const k = modelKey(title) ?? (mf ? modelKey(mf[1]) : null);
  if (!k) return "modèle d'entrée de gamme ou inconnu";
  const caps: number[] = [];
  if (s.maxPrice) caps.push(s.maxPrice);
  const c = locked ? modelLocked.get(k) : modelCap.get(k);
  if (locked && s.maxPrice) caps.splice(0, caps.length);
  if (c) caps.push(c);
  else return `modèle sans plafond (${k})`;
  const cap = Math.min(...caps);
  if (l.price > cap) return `trop cher (${l.price}€ > ${cap}€)`;

  const bad = [...config.exclude, ...(s.exclude ?? [])].find((w) => title.include
  if (bad) return `mot exclu « ${bad} »`;
  return null;
};
```

```
// ---------- Envoi ----------
let sentTotal = 0;
async function handle(items: Listing[], s: Search, silent: boolean): Promise<numb
  let fresh = 0;
  for (const l of items.reverse()) {
    if (seen.has(l.id)) continue;
    fresh++;
    const why = refusal(l, s);
    if (why) {
      if (!rejected.has(l.id)) {
        rejected.add(l.id);
        if (!silent) console.log(`🚫 ${why} | ${l.price}€ | ${l.title.slice(0, 70
      }
      continue;
    }
    seen.add(l.id);
    if (silent) continue;
    try {
      await sendToDiscord(l, s.name);
      sentTotal++;
      console.log(`➡️ ${l.source} | ${l.price}€ | ${l.title}`);
    } catch (e) {
      seen.delete(l.id); // on réessaiera au prochain passage
      console.error("Discord :", (e as Error).message);
    }
    await sleep(700);
  }
  return fresh;
}

// ⚡ Flux rapide : toutes les nouvelles annonces Samsung, tous pays configurés
const FAST_SEARCH: Search = { name: "⚡ Nouvelle annonce", query: FAST.query };
async function fastTick(silent: boolean) {
  for (const host of FAST.domains) {
    for (const q of FAST.queries) {
      try {
        const items = await searchVinted(q, FAST.priceTo, host, 1, true);
        const name = `⚡ Nouvelle annonce ${countryOf(host)}`;
        const fresh = await handle(items, { ...FAST_SEARCH, query: q, name }, sil
        // Beaucoup de nouveautés d'un coup : on lit aussi la page 2 pour ne rien
        if (!silent && fresh >= 80) {
          await sleep(1500);
          await handle(await searchVinted(q, FAST.priceTo, host, 2, true), { ...F
        }
      } catch (e) {
```

```
        console.error(`[⚡ ${countryOf(host)} ${q}] :`, (e as Error).message);
      }
      await sleep(2000);
    }
  }
}

// 🐢 Filet de sécurité : les recherches par modèle de config.json, une seule par
let slowIdx = 0;
const slowReady = new Set<number>(); // le 1er passage de chaque recherche est si
async function slowStep(silentAll: boolean) {
  const i = slowIdx++ % config.searches.length;
  const s = config.searches[i];
  const silent = silentAll || !slowReady.has(i);
  slowReady.add(i);
  const sources = [
    { name: "vinted", run: () => searchVinted(s.query, s.maxPrice, "www.vinted.fr
    ...(EBAY_ENABLED ? [{ name: "ebay", run: () => searchEbay(s.query, s.maxPrice
  ];
  for (const src of sources) {
    try {
      await handle(await src.run(), s, silent);
    } catch (e) {
      console.error(`[${src.name}] "${s.query}" :`, (e as Error).message);
    }
  }
}

console.log("📡 Samsung Scanner démarré");
try {
  await sendToDiscord(
    { id: "start", source: "vinted", title: "✅ Samsung Scanner est en ligne (Sam
      extra: `⚡ Mode rapide : nouvelle lecture toutes les ${FAST.everySeconds} s
    "Démarrage"
  );
} catch (e) {
  console.error("Discord :", (e as Error).message);
}

// Premier passage silencieux : il mémorise l'existant pour n'envoyer que les NOU
console.log("⏳ Mémorisation des annonces déjà en ligne…");
await fastTick(true);
saveSeen();
console.log("✅ Prêt : chaque nouvelle annonce arrive maintenant sur Discord");

let tick = 0;
```

```
let lastLog = Date.now();
while (true) {
  const t0 = Date.now();
  if (FAST.enabled) await fastTick(false);
  await sleep(2500);
  await slowStep(false); // une recherche par modèle à chaque tour
  if (++tick % 10 === 0) saveSeen();
  if (Date.now() - lastLog > 5 * 60_000) {
    console.log(`📊 ${sentTotal} annonces envoyées depuis le démarrage`);
    lastLog = Date.now();
  }
  const wait = FAST.everySeconds * 1000 - (Date.now() - t0);
  await sleep(Math.max(wait, 1500) + Math.floor(Math.random() * 1500)); // petit
}
```